

Since the current frame object is deleted instead of retained on the call stack, the call stack never grows in size and can never cause a stack overflow!

Recall from Chapter 1 that all recursive algorithms can be implemented with a stack and a loop. Since tail call optimization removes the need for a call stack, we are effectively using recursion to simulate a loop's iterative code. However, earlier in this book I stated that the problems suitable for recursive solutions involve a tree-like data structure and backtracking. Without a call stack, no tail recursive function could possibly do any backtracking work. In my view, every algorithm that can be implemented with tail recursion would be easier and more readable to implement with a loop instead. There's nothing automatically more elegant about using recursion for recursion's sake.